



Rapport de Projet

Distributed Memory

Hugo CLAVEILLE

Master 1 Informatique

27 décembre 2025

Table des matières

1	Introduction	3
2	Evaluation	4
2.1	Details du noeud testé	4
2.2	Details du modele d'entraînement testé	4
2.3	Impact du nombre de CPU/GPU	4
2.3.1	Impact du nombre de CPU	5
2.3.2	Impact du nombre de GPU	5
2.4	Impact de la taille du batch	6

Introduction of the cluster Grid5000 and node tested (2 points) *Introduction of the task (1 point) Evaluation (17 points) Details of the node tested (2 points) Details of the training model tested (1 point) Impact of the number of computing devices (8 points) Impact of the batch size (6 points) Test additional batch size 16,64,128 (GPU node) Three plots (for throughput) (3 points) One plot shows the optimal number of devices for different batch sizes (1 point) Analyses (2 points)

1. Introduction

Dans le cadre du projet de dynamique programming il nous a été demandé de réaliser une étude sur l'impact du nombre de ressources de calcul (CPU/GPU) et de la taille du batch sur les performances d'entraînement d'un modèle de deep learning. Pour cela nous avons utilisé le cluster Grid5000 et plus particulièrement le noeud *sophia* qui est équipé de GPU.

2. Evaluation

2.1 Details du noeud testé

2.2 Details du modele d'entraînement testé

Le modèle utilisé dans ce projet est VGG19 avec Batch Normalization (VGG19BN), issu de l'architecture VGG proposée dans l'article "Very Deep Convolutional Networks for Large-Scale Image Recognition". Il s'agit d'un réseau de neurones convolutif profond composé de 19 couches avec paramètres, incluant 16 couches convolutives et 3 couches entièrement connectées.

La variante VGG19_BN intègre des couches de Batch Normalization après chaque convolution, ce qui permet de stabiliser l'entraînement, d'accélérer la convergence et d'améliorer l'utilisation des ressources de calcul. Le modèle est implémenté à l'aide de la bibliothèque torchvision via la fonction `torchvision.models.VGG19_BN`.

VGG19_BN est un modèle particulièrement exigeant en ressources, avec environ 143 millions de paramètres et un coût de calcul d'environ 19,6 GFLOPS, ce qui en fait un candidat pertinent pour l'évaluation des performances et de la scalabilité sur des nœuds GPU. Sa complexité permet d'observer clairement l'impact du nombre de dispositifs de calcul et de la taille des batchs sur le débit d'entraînement (throughput).

2.3 Impact du nombre de CPU/GPU

Afin d'évaluer l'impact du nombre de ressources de calcul sur les performances d'entraînement, j'ai réalisé plusieurs tests en faisant varier le nombre de CPU et de GPU utilisés. J'ai aussi modifié le fichier *demo.py* pour enregistrer les temps de chargement des données et les temps de calculs dans des fichiers csv. Enfin, j'ai utilisé des scripts python pour traiter les résultats et générer la données nécessaires aux graphiques.

2.3.1 Impact du nombre de CPU

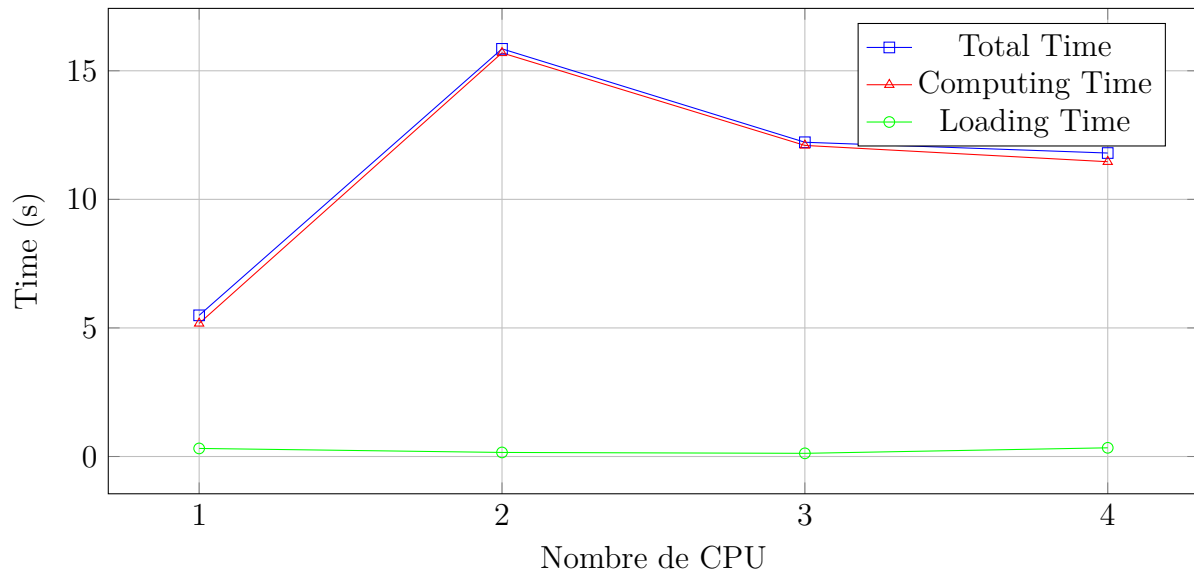


FIGURE 2.1 – Impact du nombre de CPU sur les temps d’entraînement.

On peut donc observer que sur un batch de taille donnée (32 ici), l’augmentation du nombre de CPU ne permet pas de réduire le temps d’entraînement.

2.3.2 Impact du nombre de GPU

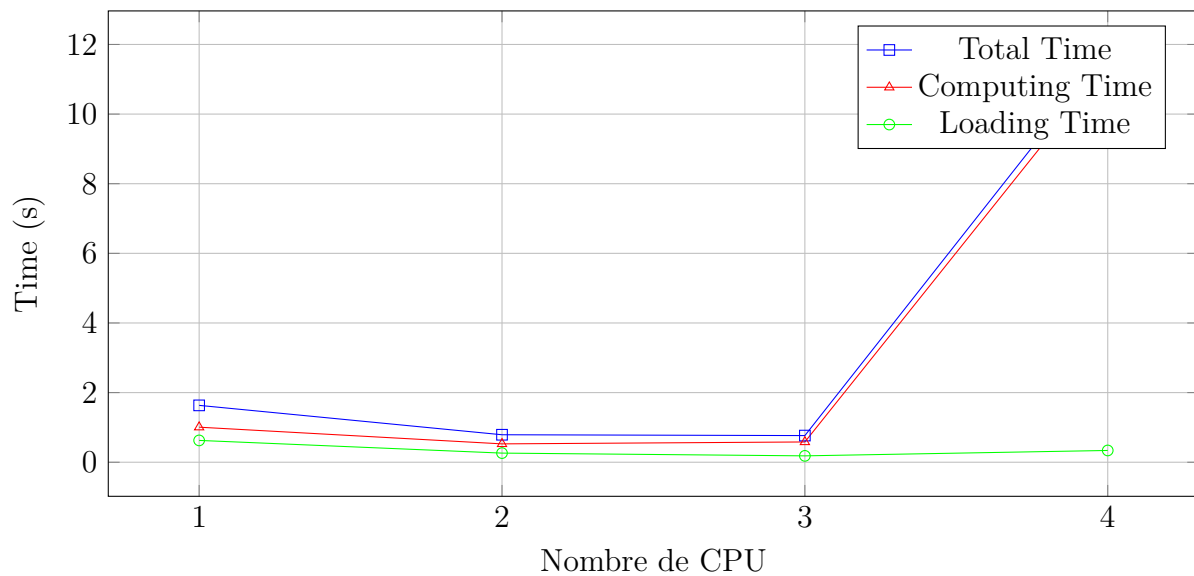


FIGURE 2.2 – Impact du nombre de GPU sur les temps d’entraînement.

On peut observer que l'augmentation du nombre de GPU permet de réduire légèrement le temps d'entraînement, mais l'impact reste limité sur ce modèle et cette configuration. A noter que le temps de chargement à partir de 4 GPU augmente significativement.

2.4 Impact de la taille du batch